

14. WeakRef aur FinalizationRegistry

Bahut rare topic hai. Naye learners skip kar sakte hain. Inhe use karna hai to bahut soch samajhke.

Pehle: Strong vs Weak Reference

- **Strong reference:** normal reference. Jab tak ek bhi strong reference hai, garbage collector (GC) object ko delete **nahi** karta.
- **Weak reference:** object ko zinda **nahi** rakhta. Agar sirf weak references bache hain, to GC use delete kar sakta hai.

WeakRef

```
js

let user = { name: "John" };
let admin = new WeakRef(user);    // weak reference

user = null;    // strong reference gaya
```

Object ab "Schrödinger's cat" jaisa hai: pata nahi zinda hai ya GC ne hata diya.

Object wapas lene ke liye `deref()`:

```
js

let ref = admin.deref();
if (ref) { /* abhi bhi memory me hai */ }
else     { /* GC ne hata diya, undefined mila */ }
```

Use case: Cache

Bade objects (images, blobs) ka cache, jisme cache ki wajah se wo memory me atke na rahein:

```
js
```

```
function weakRefCache(fetchImg) {
  const imgCache = new Map();
  return (imgName) => {
    const cached = imgCache.get(imgName);
    if (cached?.deref()) return cached.deref();

    const newImg = fetchImg(imgName);
    imgCache.set(imgName, new WeakRef(newImg));
    return newImg;
  };
}
```

- `Map` seedha use karte to objects memory me hi rehte.
- `WeakMap` ke keys weak hote hain, values nahi, isliye yaha kaam nahi karta.
- Problem: GC ke baad `Map` me khaali keys bachi rehti hain.

Use case: DOM elements track karna

Koi logger sirf tab tak messages bheje jab tak element DOM me hai. `WeakRef` se element pakdo, `deref()` `undefined` de to timer band karo.

(GC ka time hamare control me nahi hota. Chrome DevTools > Performance > "Collect garbage" se force kar sakte ho.)

FinalizationRegistry

Jab registered object GC se delete ho, tab ek **cleanup callback** chalane ke liye:

```
js

const registry = new FinalizationRegistry((heldValue) => {
  console.log(`${heldValue} collected`);
});

let user = { name: "John" };
registry.register(user, user.name);
```

- `register(target, heldValue, [unregisterToken])`
- `unregister(unregisterToken)`
- Registry object ko strong reference nahi rakhta.
- Callback **guaranteed nahi** hai. Tab band hone par ya registry khud unreachable ho to nahi chalta.

Cache ko saaf rakhna (dono saath)

js

```
const registry = new FinalizationRegistry((imgName) => {  
  const cached = imgCache.get(imgName);  
  if (cached && !cached.deref()) imgCache.delete(imgName);  
});  
// naya image aaye to: registry.register(newImg, imgName);
```

Callback me check zaruri hai ki entry dobara add to nahi ho gayi (warna zinda entry delete ho jaayegi).

Yaad rakho

- GC ka behavior predictable nahi. Isliye ye cheezein guaranteed result ke liye nahi hain.
- Zyaadatar cases me normal cache hi behtar hai.
- `WeakRef` = zinda na rakhne wala reference, `FinalizationRegistry` = delete hone par cleanup callback.